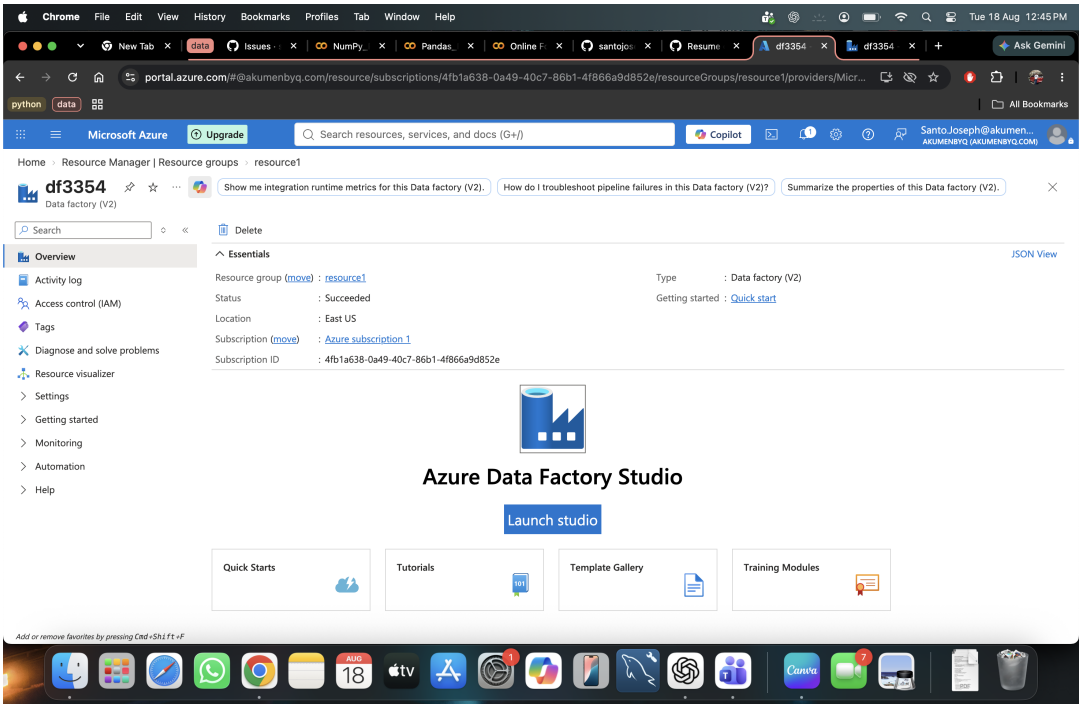
Building a Pipeline in Azure Data Factory

Calculating the area of a rectangle with Set Variable activities and pipeline parameters

This guide walks through creating a Data Factory pipeline (**df3354**) that stores values in variables, computes an area using an expression, and then upgrades the pipeline to accept the length and breadth as parameters supplied at run time.

Step 1 — Open Azure Data Factory Studio

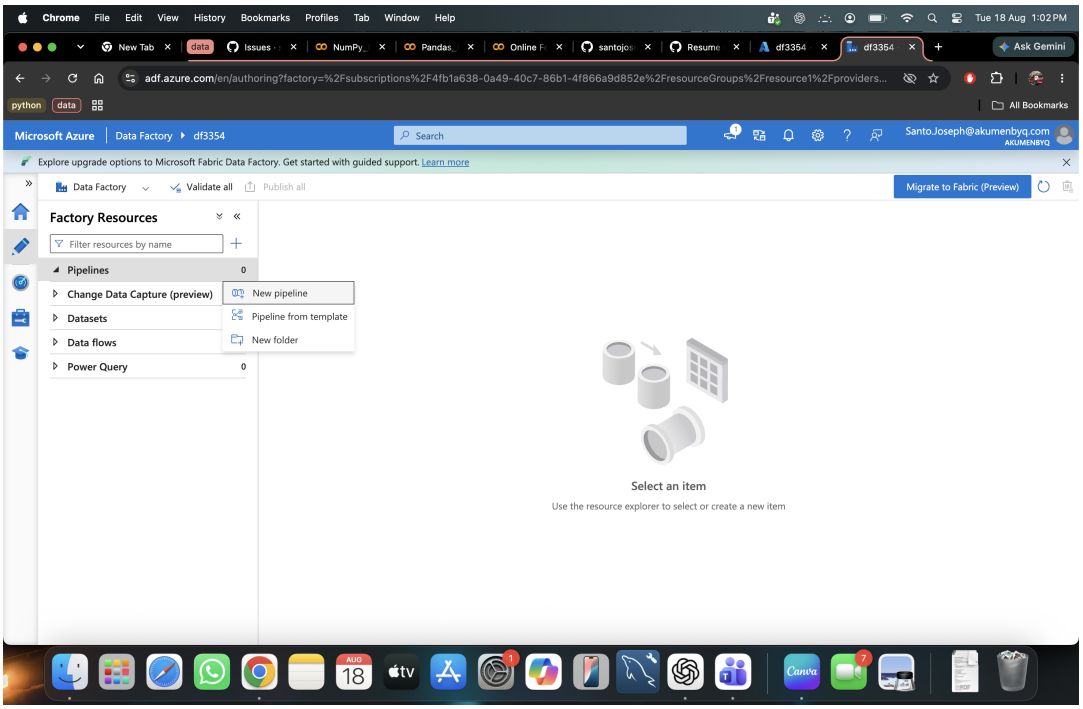
From the Data Factory resource (**df3354**) in the Azure Portal, click **Launch studio** to open Azure Data Factory Studio, where pipelines are authored.



The Data Factory overview page, with the Launch studio button.

Step 2 — Create a new pipeline

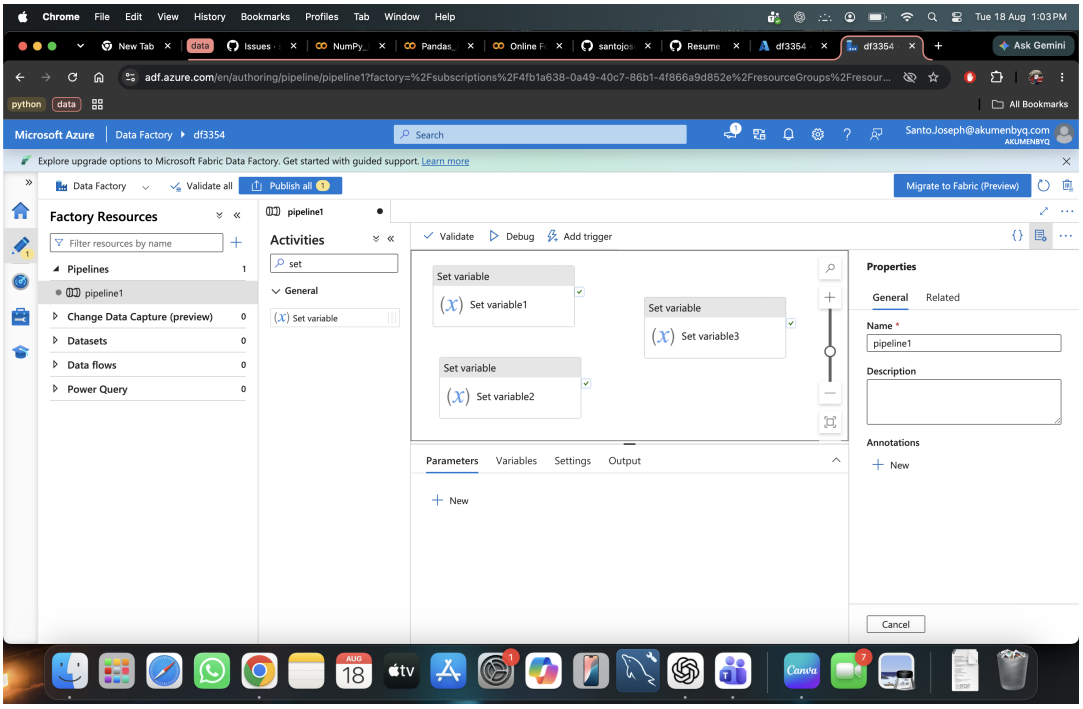
In the Author section, expand **Pipelines** in Factory Resources, click the **+** icon, and choose **New pipeline**. This opens a blank pipeline canvas (named **pipeline1**) ready for activities.



Creating a new pipeline from Factory Resources.

Step 3 — Add Set Variable activities

Search for **set** in the Activities pane and drag the **Set variable** activity onto the canvas. Three Set variable activities are added to the pipeline, one each for **length**, **breadth**, and the calculated **area**.



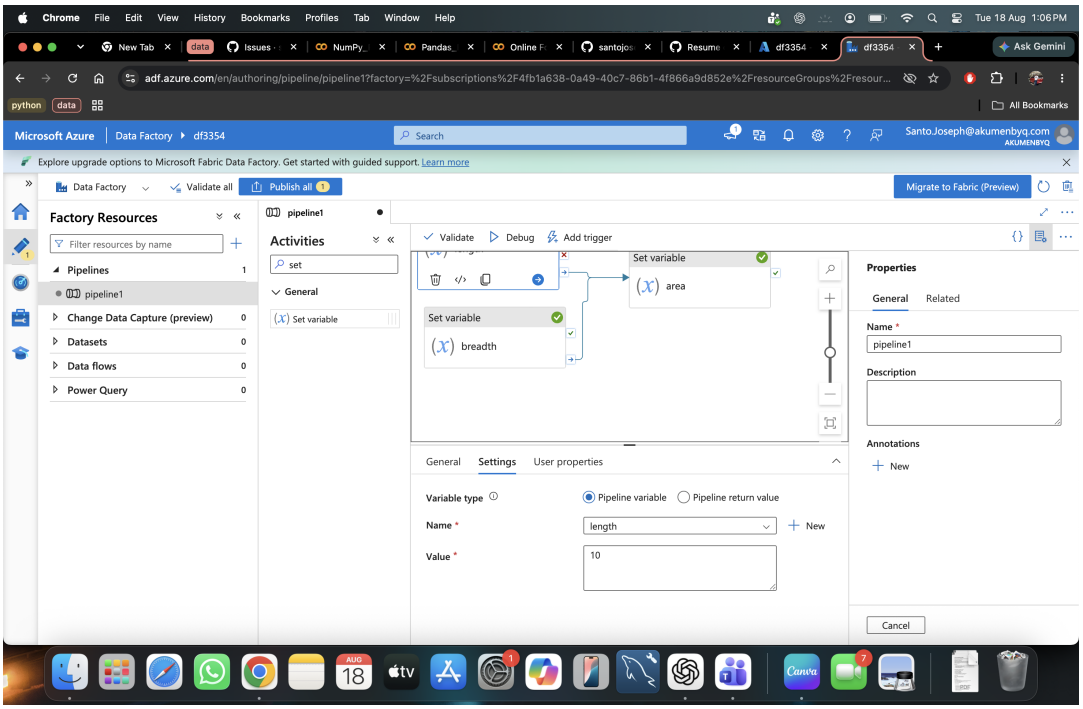
Three Set variable activities placed on the pipeline canvas.

Step 4 — Define the pipeline variables

Before an activity can set a variable, the variable itself must exist. Open the pipeline's **Variables** tab and add three pipeline variables: **length**, **breadth**, and **area** (all type Int/String as needed).

Step 5 — Configure the “length” activity

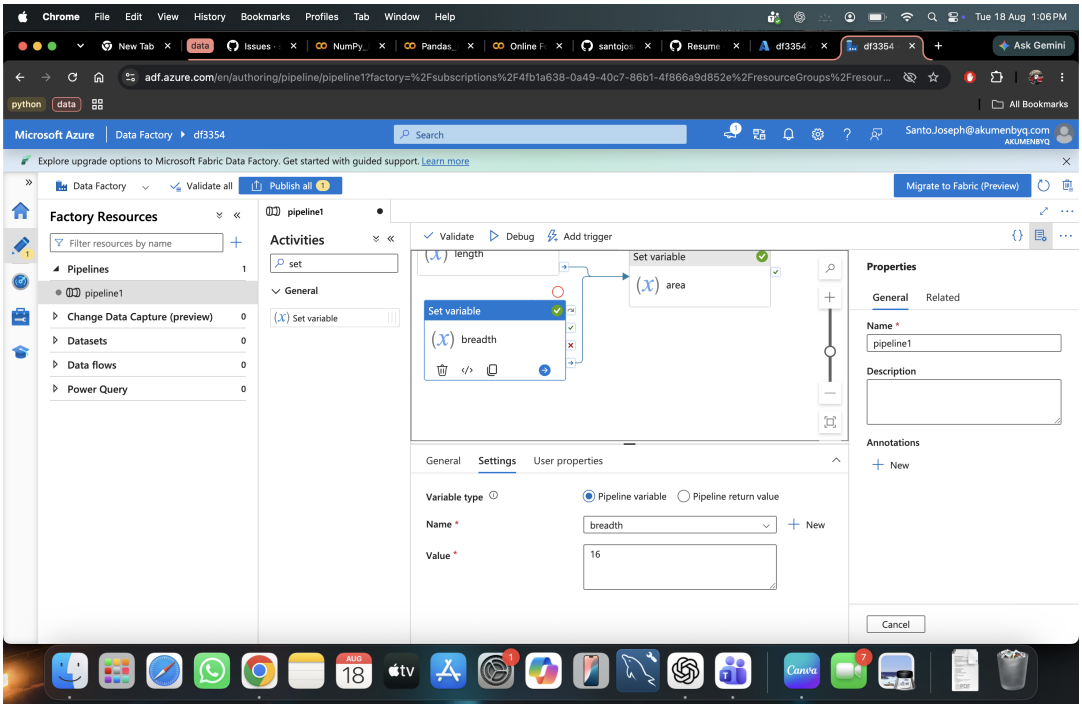
Select the first Set variable activity, open its **Settings** tab, set **Variable type** to Pipeline variable, **Name** to **length**, and **Value** to **10**.



Settings tab for the “length” Set variable activity, value = 10.

Step 6 — Configure the “breadth” activity

Select the second Set variable activity and set **Name** to **breadth** and **Value** to **16**.

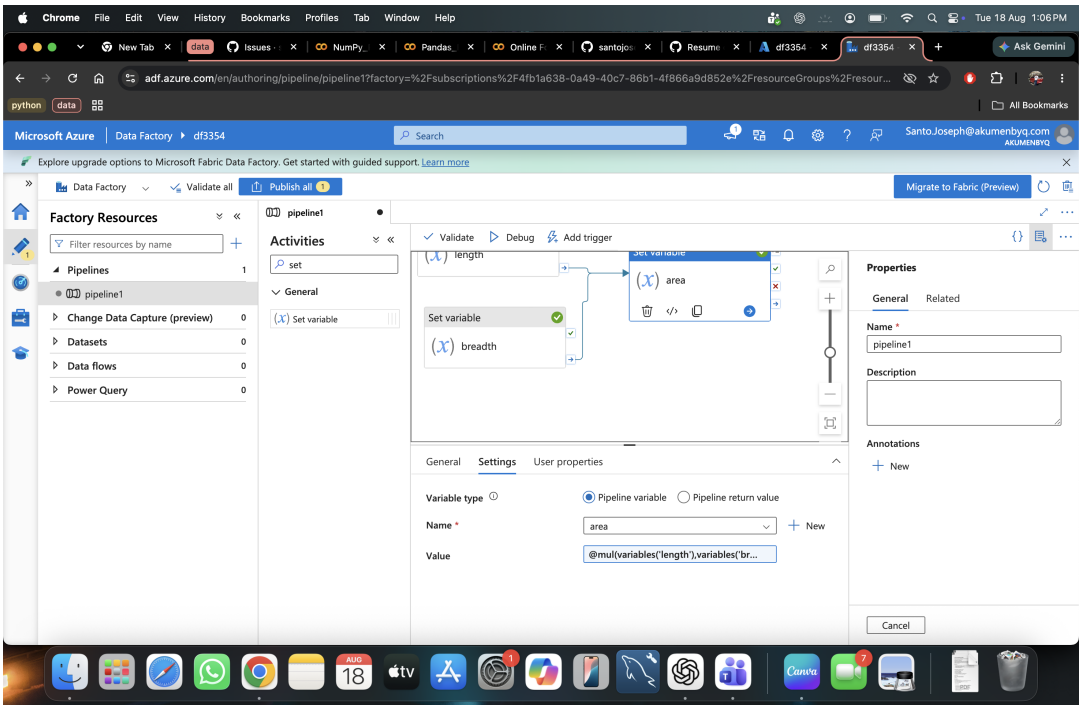


Settings tab for the “breadth” Set variable activity, value = 16.

Step 7 — Connect the activities and calculate the area

Drag the green success connector from both **length** and **breadth** into the **area** activity, so both must succeed before area runs. In the area activity's Settings, set **Name** to **area** and enter the following expression for **Value**:

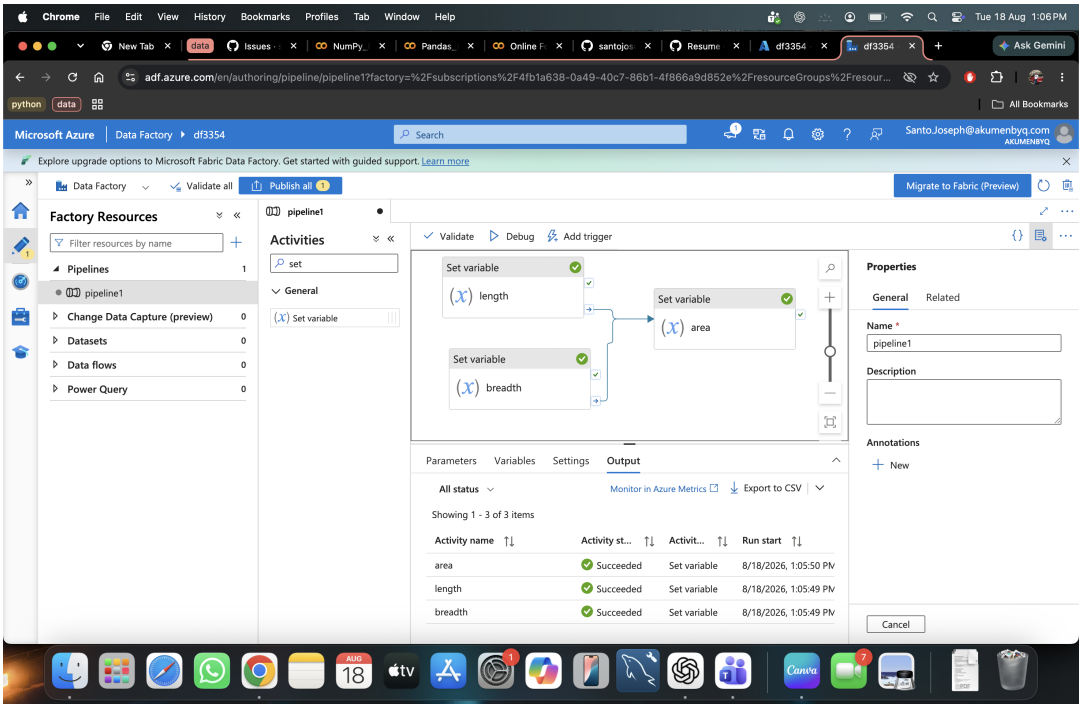
```
@mul(variables('length'),variables('breadth'))
```



The area activity wired to run after length and breadth succeed.

Step 8 — Debug the pipeline

Click **Debug** to run the pipeline without publishing. The **Output** tab lists each activity's run status — all three (length, breadth, area) show **Succeeded**.

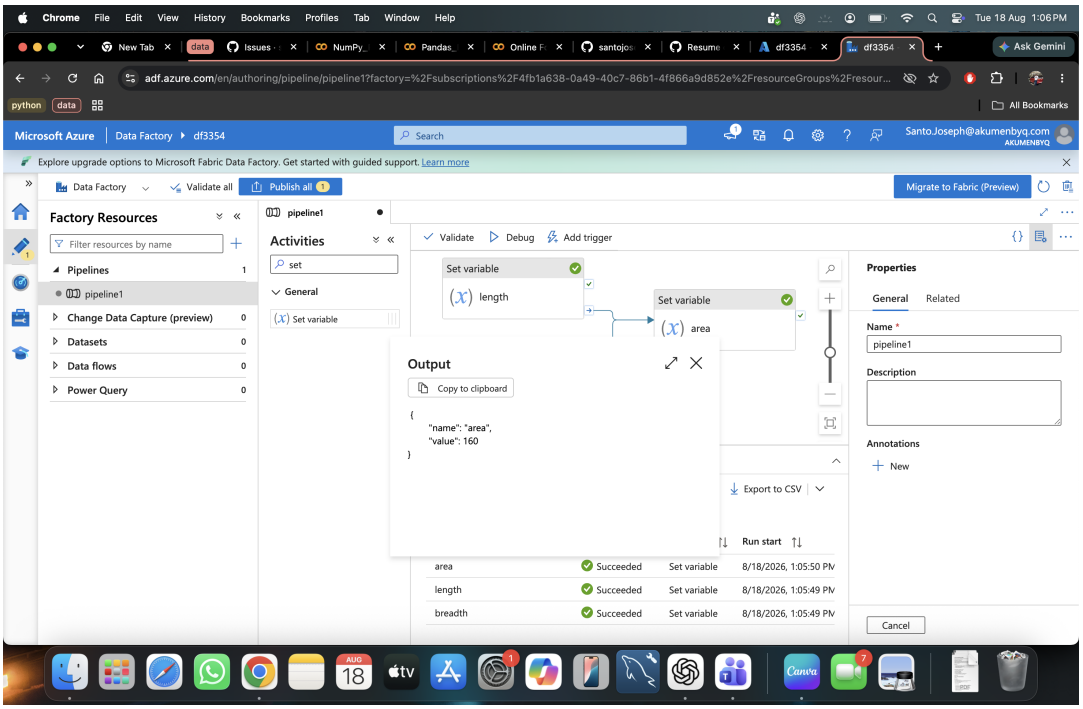


Debug run output: length, breadth, and area all succeeded.

Step 9 — Inspect the output

Click the output icon next to the **area** row to see the actual result. With length = 10 and breadth = 16, the pipeline correctly calculates:

```
{ "name": "area", "value": 160 }
```



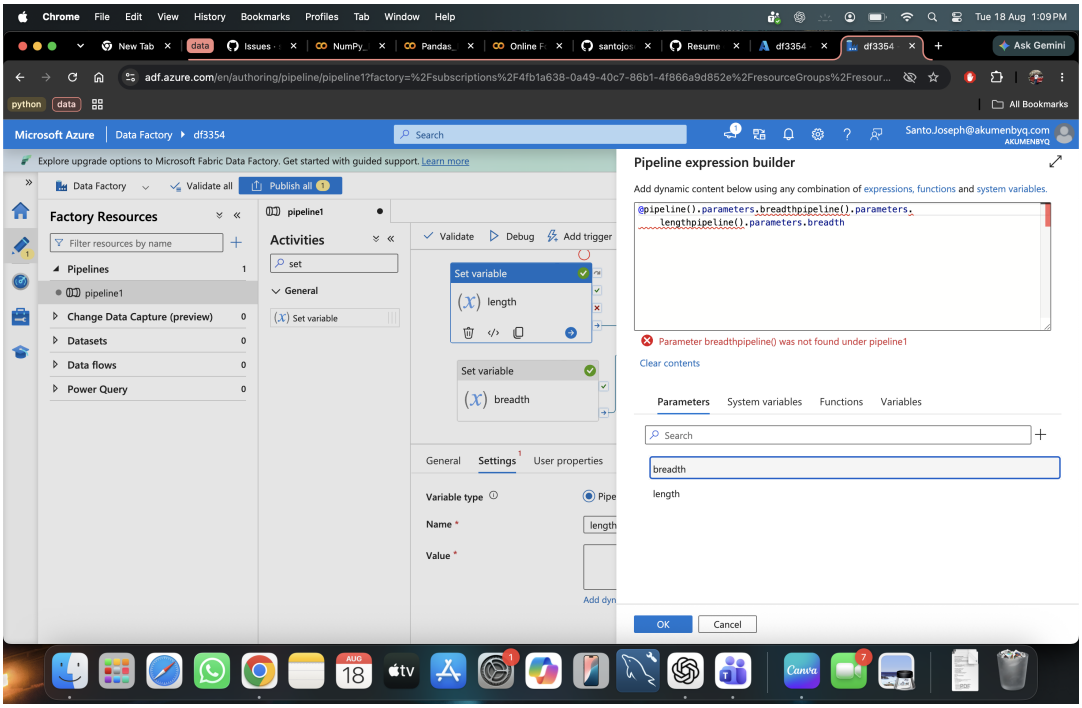
Output panel confirming area = 160 (10 × 16).

Upgrading to Pipeline Parameters

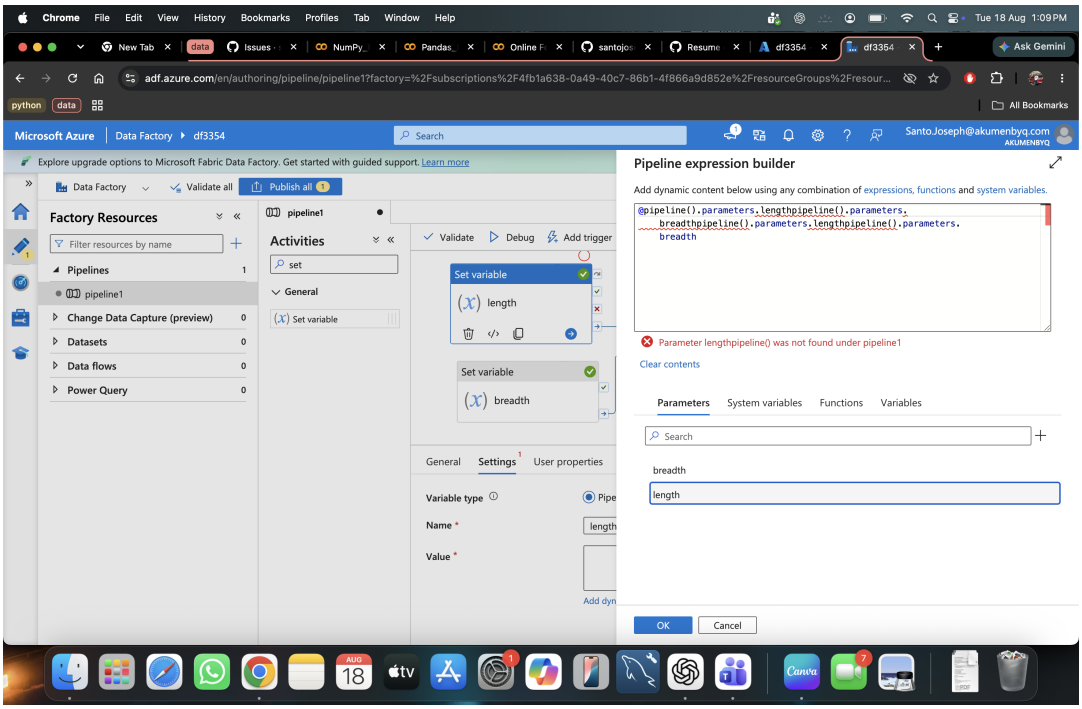
Hard-coded values only let the pipeline calculate one fixed area. To make it reusable, the next steps replace the fixed length/breadth values with **pipeline parameters** that are supplied each time the pipeline runs.

Step 10 — Reference parameters correctly

In the expression builder, parameters are referenced as **@pipeline().parameters.<parameterName>**. Two incorrect attempts are shown below — chaining **parameters** directly into another **pipeline()** call produces a “parameter was not found” validation error.



Incorrect expression: chaining pipeline().parameters.breadthpipeline() causes an error.

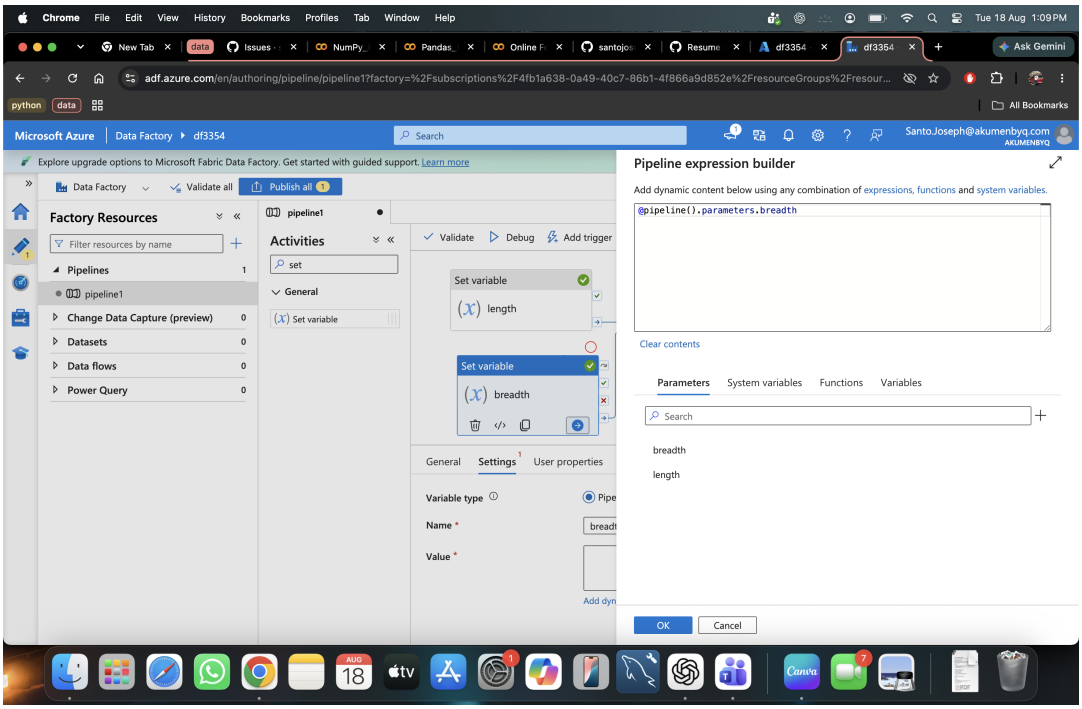


A second incorrect attempt, still referencing a non-existent lengthpipeline() parameter.

Step 11 — Correct parameter expression

The correct, simplified expression just references the parameter name directly:

```
@pipeline().parameters.breadth
```

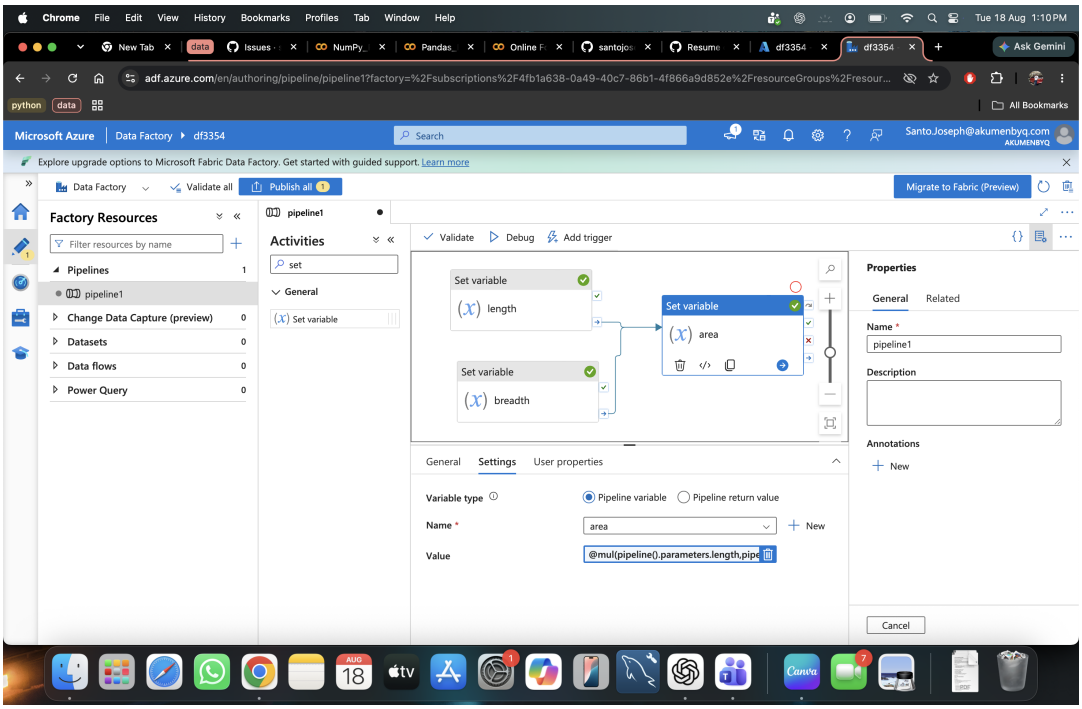


The valid expression referencing the breadth pipeline parameter.

Step 12 — Update the area expression to use parameters

With **length** and **breadth** now defined as pipeline parameters (via the Parameters tab), update the area activity's Value field to multiply the two parameters instead of the variables:

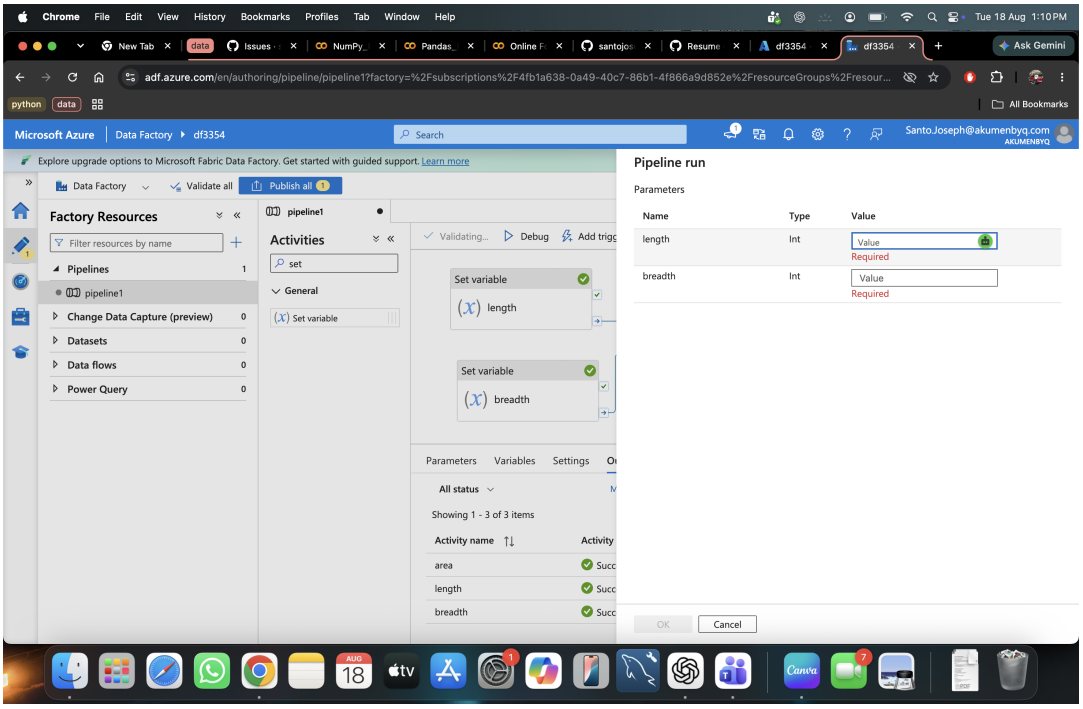
```
@mul(pipeline().parameters.length,pipeline().parameters.breadth)
```



The area activity now calculates from pipeline parameters.

Step 13 — Run with custom parameter values

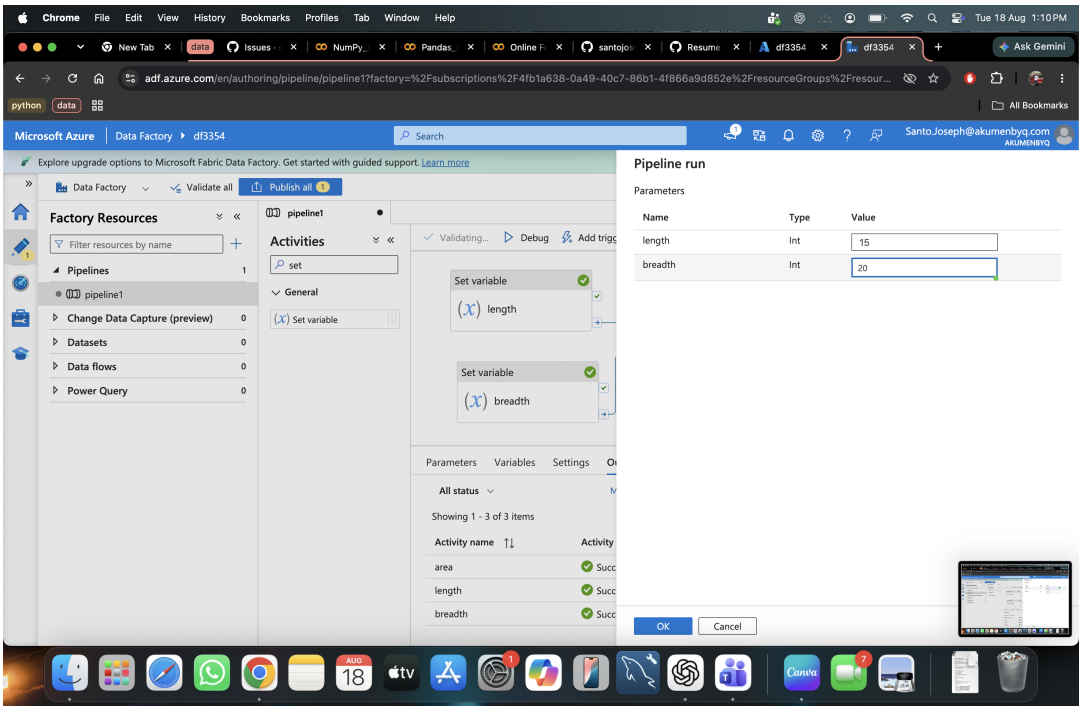
Click **Debug** again. Because the pipeline now expects parameters, a **Pipeline run** panel appears asking for **length** and **breadth** values before the run can start.



Pipeline run panel prompting for required length and breadth parameter values.

Step 14 — Enter the parameter values

Enter **length = 15** and **breadth = 20**, then click **OK** to start the run.

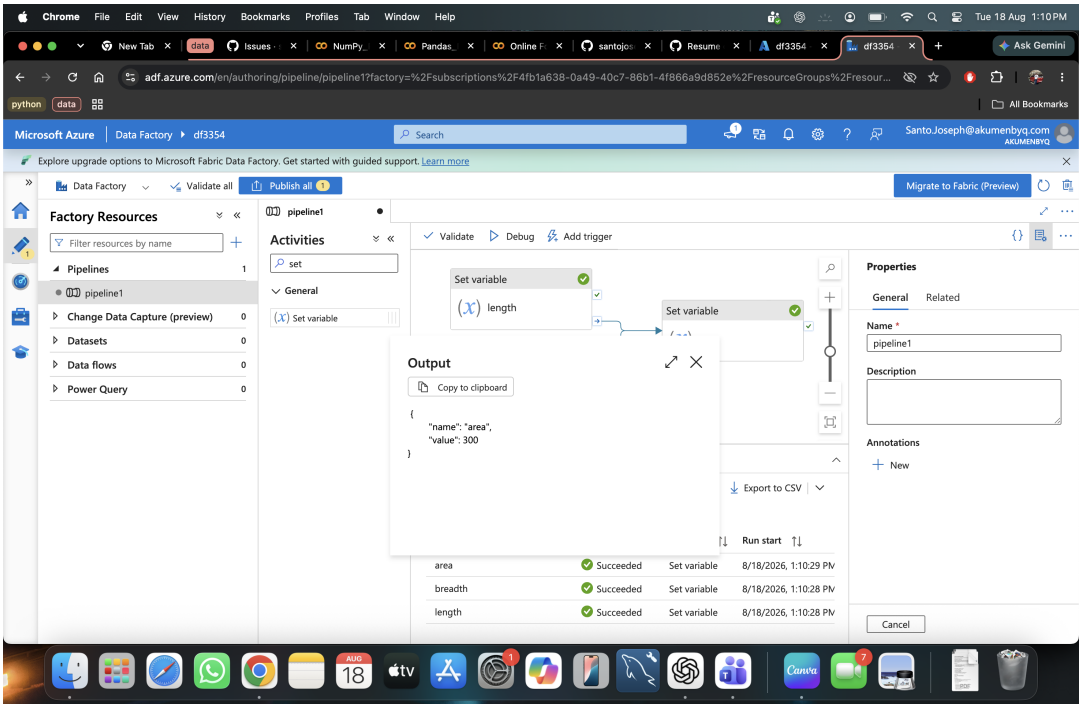


Parameter values 15 and 20 entered for this run.

Step 15 — Verify the parameterized result

The Output tab again shows all three activities succeeding. Opening the output of **area** confirms the pipeline recalculated correctly using the new inputs:

```
{ "name": "area", "value": 300 }
```



Output confirming area = 300 (15 × 20) — the pipeline is now parameter-driven.

Summary

The pipeline started with hard-coded Set Variable values (10 × 16 = 160) and was then converted to accept **length** and **breadth** as pipeline parameters, so any run can pass different inputs (e.g. 15 × 20 = 300) without editing the pipeline itself. Don't forget to **Publish all** once you're happy with the pipeline, so the changes are saved to the Data Factory.